COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/30/2026	**DATE SUBMITTED:**	09/30/2026
NAME:	Jan Mikaela Guarin Jiro Luis Katindig Juan Paulo Lara Gian Carlo G. Libo-on Ramelissa Melith V. Limpio	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 10.1 GRAPHS

1. PROCEDURE OUTPUT

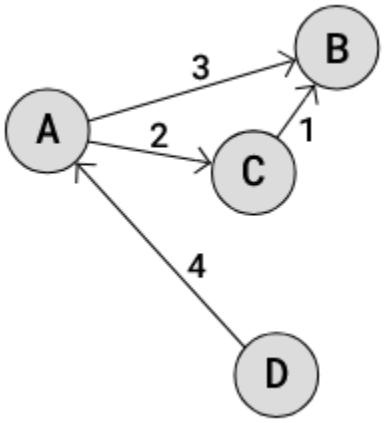
What procedures did you perform?

1. Define the following: (10)
   - A. Graph
   - A non-linear type of data structure that is made up of nodes, also called vertices, and edges that connect the vertices.
   - B. Vertex
   - A fundamental part of a graph that represents a point or entity. Each vertex can have a name called a key and can also have additional information called a payload.
   - C. Nodes
   - The elements or points in a graph. They are also called vertices and are connected to each other through edges.
   - D. Weight
   - A value associated with an edge that represents a cost of traversing from one vertex to another, such as the distance between two nodes.
   - E. Path
   - A sequence of vertices that are connected by edges. It is formally defined as w_1, w_2, w_3, …, w_n for all $1 \leq i \leq n-1$. The length of an unweighted path is the number of edges in the path, while the length of a weighted path is the number of weights of its edges.
   - F. Directed graph
   - A graph in which all edges are one-way. Each edge is an ordered pair (v, w), meaning you can go from v to w but not necessarily back. The course prerequisites graph is a digraph because some classes must be taken before others.
   - G. Connected graph
   - A graph in which there is a path between every pair of vertices, so every vertex can be reached from every other vertex. If some vertices can't be reached from others, the graph is disconnected.
   - H. Directed cyclic
   - A directed graph that contains at least one cycle. A cycle is a path that starts and ends at the same vertex, following the direction of the edges. A graph with no cycles is acyclic, and a directed graph with no cycles is a DAG (directed acyclic graph).

I. Adjacency matrix
- A two-dimensional matrix implementation of a graph. Each row and each column represents a vertex. The cell at row v and column w shows whether there is an edge from v to w. In a weighted graph, the cell holds the weight of that edge. Two vertices connected by an edge are said to be adjacent. It is simple and works well for graphs with many edges, but it wastes space on sparse graphs because most cells are empty. It needs V^2 cells for V vertices.

J. Adjacency List
- A more space-efficient implementation for sparsely connected graphs. The Graph object keeps a master list of all vertices, and each vertex keeps its own list of the vertices it is connected to. In the chapter's implementation each vertex uses a map (dictionary) whose keys are the neighboring vertices and whose values are the edge weights. It stores a sparse graph compactly and makes it easy to find all vertices directly connected to a given vertex

2. Identify the parts of the following graph. (10)



Vertices/Nodes:
- A, B, C, D

Edges:
- A -> B
- A -> C
- C -> B
- D -> A

Weights:
- A -> B = 3
- A -> C = 2
- C -> B = 1
- D -> A = 4

Paths:
- A -> B
- A -> C -> B
- D -> A -> B
- D -> A -> C -> B

Type of Graph:
- Directed weighted graph

3. Plot the adjacency matrix of the graph above. (10 pts)

	A	B	C	D
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

4. Create a program in C++ to create a graph that looks like in the above figure.

New.cpp > main()

```cpp
#include <iostream>

int main() {
    char nodes[] = {'A', 'B', 'C', 'D'};

    // 4x4 Adjacency Matrix
    int adjMtrx[4][4] = {
        {0, 3, 2, 0},
        {0, 0, 0, 0},
        {0, 1, 0, 0},
        {4, 0, 0, 0}
    };

    // Printing the matrix
    std::cout << "  ";
    for (int i = 0; i < 4; i++) {
        std::cout << " " << nodes[i];
    }
    std::cout << "\n";
    for (int i = 0; i < 4; i++) {
        std::cout << nodes[i] << " ";
        for (int j = 0; j < 4; j++) {
            std::cout << " " << adjMtrx[i][j];
        }
        std::cout << "\n";
    }

    return 0;
}
```

Figure 1. Code snippet

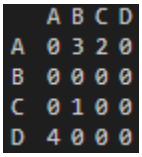


Figure 2. Output

5. Differentiate BFS vs DFS types of search.

Type	Traversal	Data Structure	Path (unweighted graph)	Target	Memory Allocation
BFS	Level by Level	Uses QUEUE (FIFO)	Finds the Shortest Path in an unweighted graph	Close to the starting position	Higher for wider graphs
DFS	Sub-tree by Sub-tree	Uses STACK (LIFO)	Not guaranteed to find the shortest path	Far from the starting position	Lower for deeper graphs

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.